

T2: Análisis Exploratorio y Plan Algorítmico

Predicción de la resistencia a compresión del concreto — Grupo 03

Objetivo del T2: verificar la calidad del conjunto de datos, documentar las decisiones de limpieza, efectuar un análisis exploratorio y definir los modelos de regresión y las métricas que se utilizarán.

Fuente oficial: Yeh, I.-C. (1998). *Concrete Compressive Strength* [Dataset]. UCI Machine Learning Repository. DOI: 10.24432/C5PK67.

El conjunto tiene **1030 observaciones**, **8 variables predictoras cuantitativas** y **1 variable objetivo cuantitativa**. La tarea es de **regresión**.

1. Librerías y configuración reproducible

```
In [1]: from pathlib import Path
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import KFold, GroupKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
from sklearn.neural_network import MLPRegressor
from sklearn.dummy import DummyRegressor
from sklearn.inspection import permutation_importance

RANDOM_STATE = 42
pd.set_option('display.max_columns', 30)
pd.set_option('display.float_format', lambda x: f'{x:,.3f}')
```

2. Carga e identificación del conjunto de datos

```
In [2]: # El notebook busca primero una copia local, lo que facilita subirlo a GitHub.
# Si no la encuentra, usa una copia pública del mismo dataset UCI.
candidates = [
    Path('concrete.csv'),
    Path('data/concrete.csv'),
    Path('/mnt/data/concrete.csv'),
]
```

```

for path in candidates:
    if path.exists():
        DATA_PATH = path
        break
else:
    DATA_PATH = 'https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-

df = pd.read_csv(DATA_PATH)
print('Fuente leída:', DATA_PATH)
print('Dimensiones:', df.shape)
df.head()

```

Fuente leída: https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/concrete.csv
 Dimensiones: (1030, 9)

```

Out[2]:
   cement  slag  ash  water  superplastic  coarseagg  fineagg  age  strength
0  540.000   0.000  0.000  162.000         2.500    1,040.000   676.000   28    79.990
1  540.000   0.000  0.000  162.000         2.500    1,055.000   676.000   28    61.890
2  332.500  142.500  0.000  228.000         0.000     932.000   594.000  270    40.270
3  332.500  142.500  0.000  228.000         0.000     932.000   594.000  365    41.050
4  198.600  132.400  0.000  192.000         0.000     978.400   825.500  360    44.300

```

Diccionario de variables

Variable	Rol	Unidad	Descripción
cement	Predictora	kg/m ³	Contenido de cemento
slag	Predictora	kg/m ³	Escoria granulada de alto horno
ash	Predictora	kg/m ³	Ceniza volante
water	Predictora	kg/m ³	Agua
superplastic	Predictora	kg/m ³	Superplastificante
coarseagg	Predictora	kg/m ³	Agregado grueso
fineagg	Predictora	kg/m ³	Agregado fino
age	Predictora	días	Edad de curado
strength	Objetivo	MPa	Resistencia a compresión del concreto

Los valores cero de escoria, ceniza volante o superplastificante representan **ausencia del componente** y no se consideran valores perdidos.

3. Auditoría de calidad y limpieza

```

In [3]: quality = pd.DataFrame({
        'tipo': df.dtypes.astype(str),
        'nulos': df.isna().sum(),
        'valores_unicos': df.nunique(),

```

```
'mínimo': df.min(),
'máximo': df.max(),
})
quality
```

Out[3]:

	tipo	nulos	valores_unicos	mínimo	máximo
cement	float64	0	278	102.000	540.000
slag	float64	0	185	0.000	359.400
ash	float64	0	156	0.000	200.100
water	float64	0	195	121.800	247.000
superplastic	float64	0	111	0.000	32.200
coarseagg	float64	0	284	801.000	1,145.000
fineagg	float64	0	302	594.000	992.600
age	int64	0	14	1.000	365.000
strength	float64	0	845	2.330	82.600

In [4]:

```
print('Total de valores nulos:', int(df.isna().sum().sum()))
print('Filas duplicadas exactas:', int(df.duplicated().sum()))
print('Registros totales:', len(df))

mix_cols = ['cement', 'slag', 'ash', 'water', 'superplastic', 'coarseagg', 'fineagg']
print('Composiciones de mezcla únicas (sin considerar edad):', df[mix_cols].drop
```

Total de valores nulos: 0

Filas duplicadas exactas: 25

Registros totales: 1030

Composiciones de mezcla únicas (sin considerar edad): 427

Decisión sobre duplicados

Se identifican filas exactamente repetidas. No se eliminan de manera automática porque pueden corresponder a réplicas experimentales válidas. Sin embargo, un reparto aleatorio podría ubicar observaciones de la misma mezcla en entrenamiento y prueba, produciendo una evaluación demasiado optimista. Para aplicar el principio de **estabilidad PCS**, se realizarán dos comprobaciones:

1. **KFold aleatorio**, comparable con gran parte de la literatura.
2. **GroupKFold por composición de mezcla**, que impide que una misma dosificación aparezca simultáneamente en entrenamiento y prueba.

Además, se compararán resultados con y sin duplicados exactos como análisis de sensibilidad.

4. Estadística descriptiva

In [5]: `df.describe().T`

Out[5]:

	count	mean	std	min	25%	50%	75%	max
cement	1,030.000	281.168	104.506	102.000	192.375	272.900	350.000	540.000
slag	1,030.000	73.896	86.279	0.000	0.000	22.000	142.950	359.400
ash	1,030.000	54.188	63.997	0.000	0.000	0.000	118.300	200.100
water	1,030.000	181.567	21.354	121.800	164.900	185.000	192.000	247.000
superplastic	1,030.000	6.205	5.974	0.000	0.000	6.400	10.200	32.200
coarseagg	1,030.000	972.919	77.754	801.000	932.000	968.000	1,029.400	1,145.000
fineagg	1,030.000	773.580	80.176	594.000	730.950	779.500	824.000	992.600
age	1,030.000	45.662	63.170	1.000	7.000	28.000	56.000	365.000
strength	1,030.000	35.818	16.706	2.330	23.710	34.445	46.135	82.600

5. Distribuciones

```
In [6]: labels = {
    'cement': 'Cemento (kg/m³)', 'slag': 'Escoria (kg/m³)',
    'ash': 'Ceniza volante (kg/m³)', 'water': 'Agua (kg/m³)',
    'superplastic': 'Superplastificante (kg/m³)',
    'coarseagg': 'Agregado grueso (kg/m³)',
    'fineagg': 'Agregado fino (kg/m³)', 'age': 'Edad (días)',
    'strength': 'Resistencia (MPa)'
}

images_dir = Path('resultados/imagenes')

fig, axes = plt.subplots(3, 3, figsize=(13, 10))
for ax, col in zip(axes.ravel(), df.columns):
    ax.hist(df[col], bins=25, edgecolor='black', linewidth=0.5)
    ax.set_title(labels[col], fontsize=10)
    ax.set_ylabel('Frecuencia')
    ax.grid(alpha=0.2)
fig.suptitle('Distribución de las variables', fontsize=14)
fig.tight_layout(rect=(0, 0, 1, 0.97))
plt.savefig(images_dir / 'distribucion_variables.png', dpi=300)
plt.show()
```

Hallazgos iniciales: la edad, la escoria, la ceniza volante y el superplastificante presentan asimetría y concentración de valores en cero. Esto es coherente con mezclas que no incorporan determinados materiales suplementarios. La resistencia se distribuye entre valores bajos y altos, con una media cercana a 36 MPa.

In []:

6. Valores atípicos mediante la regla del rango intercuartílico

```
In [7]: q1 = df.quantile(0.25)
q3 = df.quantile(0.75)
iqr = q3 - q1
outlier_mask = (df < (q1 - 1.5 * iqr)) | (df > (q3 + 1.5 * iqr))
outliers = pd.DataFrame({
    'cantidad': outlier_mask.sum(),
    'porcentaje': 100 * outlier_mask.mean()
}).sort_values('cantidad', ascending=False)
outliers
```

```
Out[7]:
```

	cantidad	porcentaje
age	59	5.728
superplastic	10	0.971
water	9	0.874
fineagg	5	0.485
strength	4	0.388
slag	2	0.194
cement	0	0.000
ash	0	0.000
coarseagg	0	0.000

Los valores extremos no se eliminarán únicamente por una regla estadística. En este dataset pueden representar edades prolongadas, mezclas de alto desempeño o dosificaciones técnicamente posibles. Se revisarán mediante análisis de sensibilidad y residuos del modelo.

7. Correlación y relaciones bivariadas

```
In [8]: corr = df.corr(numeric_only=True)
fig, ax = plt.subplots(figsize=(10, 8))
im = ax.imshow(corr.values, aspect='auto', vmin=-1, vmax=1)
ax.set_xticks(range(len(corr.columns)), [labels[c] for c in corr.columns], rotate=45)
ax.set_yticks(range(len(corr.index)), [labels[c] for c in corr.index], fontsize=12)
for i in range(corr.shape[0]):
    for j in range(corr.shape[1]):
        ax.text(j, i, f'{corr.iloc[i, j]:.2f}', ha='center', va='center', fontsize=8)
fig.colorbar(im, ax=ax, label='Correlación de Pearson')
ax.set_title('Matriz de correlación')
fig.tight_layout()
plt.savefig(images_dir / 'matriz_correlacion.png', dpi=300)
plt.show()
```

```
In [9]: corr['strength'].sort_values(ascending=False).to_frame('correlación_con_resisten
```

Out[9]:

correlación_con_resistencia	
strength	1.000
cement	0.498
superplastic	0.366
age	0.329
slag	0.135
ash	-0.106
coarseagg	-0.165
fineagg	-0.167
water	-0.290

```
In [10]: fig, axes = plt.subplots(2, 2, figsize=(12, 9))
for ax, col in zip(axes.ravel(), ['cement', 'water', 'age', 'superplastic']):
    ax.scatter(df[col], df['strength'], alpha=0.55, s=16)
    ax.set_xlabel(labels[col])
    ax.set_ylabel(labels['strength'])
    ax.set_title(f'Resistencia vs. {labels[col].split(" ")[0]}')
    ax.grid(alpha=0.2)
fig.tight_layout()
plt.savefig(images_dir / 'resistencia_vs_variables.png', dpi=300)
plt.show()
```

La correlación lineal muestra una asociación positiva moderada de la resistencia con el cemento, el superplastificante y la edad, y una asociación negativa con el agua. No obstante, estas correlaciones no capturan interacciones ni relaciones no lineales, por lo que se justifican modelos como Random Forest, SVR con kernel RBF y redes neuronales.

8. Ingeniería de variables con fundamento técnico

```
In [11]: def add_engineered_features(X: pd.DataFrame) -> pd.DataFrame:
    X = X.copy()
    X['binder'] = X['cement'] + X['slag'] + X['ash']
    X['water_binder'] = X['water'] / X['binder']
    X['water_cement'] = X['water'] / X['cement']
    X['aggregate_total'] = X['coarseagg'] + X['fineagg']
    X['aggregate_binder'] = X['aggregate_total'] / X['binder']
    X['sp_binder'] = X['superplastic'] / X['binder']
    X['age_log'] = np.log1p(X['age'])
    return X

X_raw = df.drop(columns='strength')
y = df['strength']
X_eng = add_engineered_features(X_raw)
X_eng.head()
```

Out[11]:

	cement	slag	ash	water	superplastic	coarseagg	fineagg	age	binder	wa
0	540.000	0.000	0.000	162.000	2.500	1,040.000	676.000	28	540.000	
1	540.000	0.000	0.000	162.000	2.500	1,055.000	676.000	28	540.000	
2	332.500	142.500	0.000	228.000	0.000	932.000	594.000	270	475.000	
3	332.500	142.500	0.000	228.000	0.000	932.000	594.000	365	475.000	
4	198.600	132.400	0.000	192.000	0.000	978.400	825.500	360	331.000	

Estas variables agregan conocimiento de ingeniería y constituyen el aporte que evita una simple réplica de estudios previos: relación agua/cemento, relación agua/material cementante, total de material cementante, relación agregado/material cementante, dosificación relativa de superplastificante y transformación logarítmica de la edad.

9. Plan algorítmico

Se propone comparar los siguientes modelos:

1. **Regresión lineal múltiple:** línea base interpretable. Permitirá cuantificar cuánto mejora un método no lineal.
2. **Random Forest Regressor:** adecuado para interacciones y no linealidades, robusto frente a escalas diferentes y útil para importancia de variables.
3. **Support Vector Regression con kernel RBF:** eficaz en conjuntos de tamaño moderado y relaciones no lineales; requiere estandarización.
4. **Red neuronal multicapa (MLP):** permite comparar con el enfoque seminal de Yeh y modelar relaciones complejas; requiere estandarización, regularización y parada temprana.

La selección final se basará en validación cruzada, no únicamente en el ajuste sobre entrenamiento.

```
In [12]: models = {
    'Promedio (Dummy)': DummyRegressor(strategy='mean'),
    'Regresión lineal': Pipeline([
        ('scale', StandardScaler()),
        ('model', LinearRegression())
    ]),
    'Random Forest': RandomForestRegressor(
        n_estimators=150,
        max_features=1.0,
        random_state=RANDOM_STATE,
        n_jobs=1
    ),
    'SVR-RBF': Pipeline([
        ('scale', StandardScaler()),
        ('model', SVR(C=100, gamma='scale', epsilon=0.1, kernel='rbf'))
    ]),
}

# Modelo planificado para la etapa de ajuste final:
```

```
mlp_model = Pipeline([
    ('scale', StandardScaler()),
    ('model', MLPRegressor(
        hidden_layer_sizes=(64, 32),
        activation='relu',
        alpha=0.001,
        early_stopping=True,
        max_iter=1000,
        random_state=RANDOM_STATE
    ))
])
```

10. Métricas y esquemas de validación

Las métricas principales serán:

- **R²**: proporción de variabilidad explicada; cuanto más cercano a 1, mejor.
- **RMSE (MPa)**: penaliza con mayor intensidad los errores grandes.
- **MAE (MPa)**: error absoluto medio, fácil de interpretar en unidades de resistencia.

No se utilizará MAPE como métrica principal porque la resistencia mínima es baja y los porcentajes pueden sobredimensionar errores en observaciones pequeñas.

```
In [13]: mix_cols = ['cement', 'slag', 'ash', 'water', 'superplastic', 'coarseagg', 'fineagg']
groups = pd.util.hash_pandas_object(X_raw[mix_cols].round(6), index=False).value

cv_random = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
cv_group = GroupKFold(n_splits=5)
scoring = {
    'R2': 'r2',
    'MAE': 'neg_mean_absolute_error',
    'RMSE': 'neg_root_mean_squared_error'
}
```

11. Benchmark preliminar reproducible

```
In [14]: def evaluate(models, X, y, cv, groups=None, validation_name=''):
    rows = []
    for name, model in models.items():
        kwargs = {'groups': groups} if groups is not None else {}
        scores = cross_validate(
            model, X, y, cv=cv, scoring=scoring, n_jobs=1, **kwargs
        )
        rows.append({
            'Modelo': name,
            'Validación': validation_name,
            'R2': scores['test_R2'].mean(),
            'R2_sd': scores['test_R2'].std(),
            'MAE': -scores['test_MAE'].mean(),
            'RMSE': -scores['test_RMSE'].mean(),
        })
    return pd.DataFrame(rows)

res_random = evaluate(models, X_eng, y, cv_random, validation_name='KFold aleato
res_group = evaluate(models, X_eng, y, cv_group, groups=groups, validation_name=
```



```
results = pd.concat([res_random, res_group], ignore_index=True)
results.sort_values(['Validación', 'R2'], ascending=[True, False])
```

Out[14]:

	Modelo	Validación	R2	R2_sd	MAE	RMSE
6	Random Forest	GroupKFold por mezcla	0.896	0.021	3.791	5.331
7	SVR-RBF	GroupKFold por mezcla	0.882	0.017	3.903	5.679
5	Regresión lineal	GroupKFold por mezcla	0.814	0.025	5.528	7.121
4	Promedio (Dummy)	GroupKFold por mezcla	-0.019	0.021	13.511	16.729
2	Random Forest	KFold aleatorio	0.912	0.009	3.409	4.952
3	SVR-RBF	KFold aleatorio	0.909	0.013	3.173	5.013
1	Regresión lineal	KFold aleatorio	0.825	0.009	5.462	6.965
0	Promedio (Dummy)	KFold aleatorio	-0.001	0.001	13.467	16.671

```
In [15]: for metric in ['R2', 'RMSE']:
    pivot = results.pivot(index='Modelo', columns='Validación', values=metric)
    ax = pivot.plot(kind='bar', figsize=(10, 5), rot=20)
    ax.set_title(f'Comparación preliminar según {metric}')
    ax.set_ylabel(metric if metric == 'R2' else 'RMSE (MPa)')
    ax.grid(axis='y', alpha=0.2)
    plt.savefig(images_dir / f'comparacion_modelos_{metric.lower()}.png', dpi=30)
    plt.tight_layout()
    plt.show()
```

Los resultados preliminares sirven para comprobar que el problema contiene señal predictiva y que la validación por mezcla es más exigente. No deben presentarse todavía como resultados definitivos: en la siguiente etapa se ajustarán hiperparámetros, se incorporará la red neuronal y se evaluará la estabilidad con y sin duplicados exactos.

12. Interpretabilidad y análisis adicional propuesto

```
In [16]: # Ejemplo de importancia por permutación con una partición simple.
# En el trabajo final se calculará dentro de un protocolo de validación para evi
rf_demo = RandomForestRegressor(n_estimators=150, random_state=RANDOM_STATE, n_j
rf_demo.fit(X_eng, y)
imp = permutation_importance(rf_demo, X_eng, y, n_repeats=10, random_state=RANDO
importance = pd.DataFrame({
    'variable': X_eng.columns,
    'importancia_media': imp.importances_mean,
    'desviación': imp.importances_std
}).sort_values('importancia_media', ascending=False)
importance.head(12)
```

Out[16]:

	variable	importancia_media	desviación
9	water_binder	0.450	0.018
14	age_log	0.223	0.009
7	age	0.196	0.008
10	water_cement	0.109	0.007
8	binder	0.070	0.001
0	cement	0.028	0.001
2	ash	0.016	0.001
1	slag	0.011	0.001
6	fineagg	0.010	0.001
5	coarseagg	0.008	0.000
11	aggregate_total	0.007	0.001
4	superplastic	0.007	0.000

Aporte diferencial del proyecto

- Comparar datos originales frente a variables derivadas con fundamento de tecnología del concreto.
- Evaluar generalización a mezclas no vistas mediante GroupKFold.
- Analizar estabilidad ante la eliminación o conservación de duplicados exactos.
- Reportar importancia por permutación y, si se dispone de la librería, valores SHAP.
- Examinar residuos por grupos de edad (1–7, 8–28, 29–90 y >90 días) y por rangos de resistencia.
- Evitar extrapolar fuera de los rangos observados en el dataset.

13. Resultados previstos y criterios de éxito

Como referencia, la literatura previa reporta desempeños altos para métodos no lineales. El proyecto considerará satisfactorio un modelo que, en validación cruzada aleatoria, alcance aproximadamente $R^2 \geq 0.85$, $RMSE \leq 7 \text{ MPa}$ y $MAE \leq 5 \text{ MPa}$. La evaluación GroupKFold será reportada por separado y tendrá prioridad para valorar la generalización a nuevas dosificaciones. La comparación se realizará mediante promedio y dispersión entre pliegues, no con una sola partición.

14. Conclusiones del T2

1. El dataset está identificado y documentado: UCI Concrete Compressive Strength, 1030 registros, ocho predictores y una variable objetivo.
2. No existen valores nulos. Los ceros en materiales suplementarios son valores válidos.

- 3. Se encontraron duplicados exactos y mezclas repetidas; por ello se utilizará validación agrupada para controlar posible fuga de información.
- 4. La resistencia presenta relaciones lineales moderadas con cemento, agua, edad y superplastificante, pero la estructura general es no lineal.
- 5. El plan compara una línea base lineal con Random Forest, SVR-RBF y una MLP, evaluados con R^2 , RMSE y MAE.
- 6. El aporte diferencial será la combinación de variables ingenieriles, validación por mezcla, análisis de estabilidad e interpretabilidad.

15. Referencias

- Chou, J.-S., Chiu, C.-K., Farfoura, M., & Al-Taharwa, I. (2011). Optimizing the prediction accuracy of concrete compressive strength based on a comparison of data-mining techniques. *Journal of Computing in Civil Engineering*, 25(3), 242–253. [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000088](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000088)
- Yeh, I.-C. (1998). Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12), 1797–1808. [https://doi.org/10.1016/S0008-8846\(98\)00165-3](https://doi.org/10.1016/S0008-8846(98)00165-3)
- Yeh, I.-C. (1998). *Concrete Compressive Strength* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5PK67>
- Yu, B., & Barter, R. L. (2024). *Veridical Data Science: The Practice of Responsible Data Analysis and Decision Making*. MIT Press.
- Yu, B., & Kumbier, K. (2020). Veridical data science. *Proceedings of the National Academy of Sciences*, 117(8), 3920–3929. <https://doi.org/10.1073/pnas.1901326117>

16. Bitácora de uso de IA para esta versión

Fecha	Herramienta	Solicitud resumida	Resultado utilizado	Verificación humana
14/07/2026	ChatGPT	Organizar las observaciones del docente y estructurar el T2	Índice formal, identificación del dataset, EDA, plan algorítmico y métricas	Se contrastó con la rúbrica y la página oficial de UCI
14/07/2026	ChatGPT + Python	Elaborar y ejecutar código de calidad, estadística, correlaciones y visualizaciones	Notebook reproducible; 1030 registros, 0 nulos y detección de duplicados	Se ejecutó el código y se revisaron tablas y gráficos
14/07/2026	ChatGPT + scikit-learn	Proponer modelos y validación estable	Regresión lineal, Random Forest, SVR, MLP; KFold y GroupKFold	Se compararon métricas preliminares y se documentó que no son resultados finales

Fecha	Herramienta	Solicitud resumida	Resultado utilizado	Verificación humana
14/07/2026	ChatGPT	Redactar referencias y aporte diferencial	Bibliografía APA y propuesta de ingeniería de variables/interpretabilidad	Se verificaron DOI y metadatos en fuentes oficiales

Nota: el equipo debe completar esta bitácora con cualquier uso adicional de IA, indicar qué partes fueron modificadas por sus integrantes y conservar los prompts reales empleados.